MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on Laboratory Work no. 2
Operating Systems
Theme: Meet the OS You Will Build

Done by: st. gr. FAF-242, **Moroșanu Ion**
Checked by: **Reițman Patricia**, *university assistant*
FCIM, UTM

Chișinău – 2026

TABLE OF CONTENTS

INTRODUCTION **3**
Purpose of the work 3
1 BUILDING AND BOOTING XV6 ..... **4**
1.1 Preparing the tools 4
1.2 Building and booting 4
2 USING XV6 AS A UNIX SYSTEM **6**
2.1 The programs of xv6 6
2.2 Answers to the questions 7
3 READING THE SOURCE CODE **8**
3.1 The user programs 8
3.2 The system calls in the kernel 9
3.3 Answers to the questions 10
4 WRITING THE PROGRAM SLEEP **11**
4.1 The program and its line in the Makefile 11
4.2 The renamed system call 12
4.3 Testing sleep inside xv6 13
4.4 The path of the system call 14
CONCLUSIONS **16**
REFERENCES **17**

INTRODUCTION

Purpose of the work

The purpose of the second laboratory work is to build a real operating system from its source code, to boot it, to use it, to read its code on both sides of the system call boundary and to add a first program to it. The system is xv6, a small Unix written at MIT for teaching. It reimplements Unix Version 6 in about 9,000 lines of C for the RISC-V architecture and runs under the QEMU emulator [1].

The following tasks were set for the work:

a) to build the kernel and the user programs and to boot xv6 in QEMU;

b) to use the programs that come with xv6, including a pipe;

c) to read user programs and to find where the kernel implements the system calls they use;

d) to write a new user program, sleep, that pauses for a given number of clock ticks through a system call.

The work was done on the same Ubuntu 26.04 virtual machine as the first laboratory work. The current version of xv6 was cloned from its official repository into ~/xv6-riscv (commit 06aad25 of 26 September 2026) [2]. The program user/sleep.c, the Makefile with its UPROGS line and notes.md with the answers to the questions are in the folder lab2/ of the repository.

1 BUILDING AND BOOTING XV6

1.1 Preparing the tools

According to the handout, the cross compiler and QEMU should already be installed on the course machine, but on this virtual machine they were missing, and so was git, which was installed first to clone the repository. The first `make qemu` then failed. The Makefile did not find a compiler for riscv64 and fell back to the ordinary gcc, which builds code for x86_64 and rejected the option `-march=rv64gc`. The program `qemu-system-riscv64` did not exist either, so the Makefile could not read its version and printed two more errors.

xv6 cannot be compiled by the usual compiler of the machine it is built on, because that compiler produces x86_64 instructions and xv6 runs on a RISC-V processor. It needs a cross compiler, which runs on one architecture and produces code for another. The tools were installed with `sudo apt install qemu-system-riscv gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu gdb-multiarch`. On Ubuntu 26.04 the RISC-V emulator is in the package qemu-system-riscv, while the package qemu-system-misc named in the handout no longer contains it. After the installation the cross compiler riscv64-linux-gnu-gcc 15.2.0 and QEMU 10.2.1 were available (Figure 1.1).

```
wancea@OS-Lab1-2:~/xv6-riscv$ riscv64-linux-gnu-gcc --version
qemu-system-riscv64 --version
riscv64-linux-gnu-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

QEMU emulator version 10.2.1 (Debian 1:10.2.1+ds-1ubuntu3.2)
Copyright (c) 2003-2025 Fabrice Bellard and the QEMU Project developers
```

Figure 1.1 – Versions of the RISC-V cross compiler and of QEMU

1.2 Building and booting

The command `make qemu` compiles every file of the kernel and of the user programs with the cross compiler and links each user program into a separate executable whose name starts with an underscore, for example user/_sync. Then it runs the tool mkfs, which writes these programs and the README file into the disk image fs.img (Figure 1.2). mkfs reported an image of 2000 blocks of 1024 bytes. 47 of them hold the metadata of the file system: the boot block, the superblock, 31 log blocks, 13 inode blocks and 1 bitmap block. The other 1953 blocks are for data, and after the programs were written the first 983 blocks of the image were in use.

QEMU then starts a virtual RISC-V computer of the type virt with 128 MB of memory and 3 processor cores, which RISC-V calls harts. The option `-bios none` means that no firmware runs first, so the xv6 kernel is the first code that the virtual processor executes. The kernel printed "xv6

kernel is booting", the two other cores reported "hart 1 starting" and "hart 2 starting", and the first user process, init, started the shell sh, which showed its prompt $. QEMU runs with the option `-nographic`, so the console of xv6 is the same terminal window, and QEMU is closed with Ctrl-A followed by X.

```
sync.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_sync user/sy
nc.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_sync > user/sync.asm
riscv64-linux-gnu-objdump -t user/_sync | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$
/d' > user/sync.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_ini
t user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user
/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user
/_dorphan user/_sync
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1
953 total 2000
balloc: first 983 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp
3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,f
ormat=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

Figure 1.2 – The end of the first build and the boot of xv6

2 USING XV6 AS A UNIX SYSTEM

2.1 The programs of xv6

At the xv6 prompt the commands `cat README`, `echo hello xv6`, `ls | grep c` and `wc README` were run (Figure 2.1). They behave like their Linux versions from the first laboratory work. The pipe `ls | grep c` kept only the lines of ls that contain the letter c: cat, echo, wc, sync and console. Every line of ls shows the name, the type, the inode number and the size in bytes. The type is 1 for a directory, 2 for a file and 3 for a device, so console is a device file, just like /dev/console on Linux. The command `wc README` counted 48 lines, 336 words and 2441 bytes, the same size that ls shows for README.

```
Please send errors and suggestions to Frans Kaashoek and Robert Morris
(kaashoek,rtm@mit.edu).  The main purpose of xv6 is as a teaching
operating system for MIT's 6.1810, so we are more interested in
simplifications and clarifications than new features.

BUILDING AND RUNNING XV6

You will need a RISC-V "newlib" tool chain from
https://github.com/riscv/riscv-gnu-toolchain, and qemu compiled for
riscv64-softmmu.  Once they are installed, and in your shell
search path, you can run "make qemu".
$ echo hello xv6
hello xv6
$ echo hello xv6
hello xv6
$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 23 0
$ wc README
48 336 2441 README
$
```

Figure 2.1 – Running the programs of xv6, including a pipe

xv6 comes with 20 programs, which can be seen in the mkfs line of Figure 1.2. Three of them are cat, grep and wc, and the others include echo, ls, mkdir, rm, kill and the shell sh.

A pipe needs two features of the operating system. The first one is processes: the shell creates a separate process for each side of the pipe with fork, loads ls into one and grep into the other with exec, and the two run at the same time. The second one is communication between processes: the system call pipe creates a buffer inside the kernel with a write end and a read end, both seen by the programs as file descriptors. The shell connects the standard output of ls to the write end and the standard input of grep to the read end with close and dup, as lines 103 to 114 of user/sh.c show. Neither program knows about the other: ls writes and grep reads as if they used the terminal.

A mistake in this part showed how small xv6 is. The command `sed -n "1,40p" user/ls.c` of the next part was first typed at the xv6 prompt instead of the Linux one, and xv6 answered "exec sed failed". xv6 has no program called sed, and the source files are not on its disk at all, because mkfs copies only the compiled programs and README. The source code can be read only from Linux.

2.2 Answers to the questions

1. Three programs that come with xv6 are cat, grep and wc. The others include echo, ls, mkdir, rm, kill and the shell sh.

2. A pipe needs processes, so that the shell can run ls and grep at the same time in two processes made with fork and exec, and communication between processes, the system call pipe, a kernel buffer with a write end and a read end that the shell connects to the output of one program and the input of the other.

3. The xv6 shell understands the same basic syntax as the bash shell of the first laboratory work, with commands, arguments, pipes, redirection and background jobs, but it is a single file of 499 lines, user/sh.c, without command history, tab completion, variables, wildcards or scripts.

3 READING THE SOURCE CODE

3.1 The user programs

The xv6 tree is divided into two main folders, and this division is the main idea of how an operating system is organised. The folder kernel/ contains the operating system itself: the scheduler, the memory manager, the file system and the handlers of the system calls. The folder user/ contains the ordinary programs that run on top of it, such as ls, cat, echo and the shell. After QEMU was closed, the beginnings of user/ls.c and user/cat.c were printed with sed (Figure 3.1). They are ordinary C programs, but they include kernel/types.h and user/user.h instead of the headers of the standard C library, because xv6 has no standard library.

```
wancea@OS-Lab1-2:~/xv6-riscv$ sed -n '1,30p' user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if (argc <= 1) {
    cat(0);
wancea@OS-Lab1-2:~/xv6-riscv$
```

Figure 3.1 – The beginning of user/cat.c

The program cat uses five system calls. open asks the kernel to open a file named on the command line for reading and to return a file descriptor, a small number that refers to the open file. read asks the kernel to copy up to 512 bytes from the file into the buffer buf. write asks it to copy these bytes to file descriptor 1, the standard output. close releases the descriptor when the file is finished, and exit ends the process with a status, 0 for success and 1 for an error. The error messages printed with fprintf also end up as write calls on file descriptor 2. When cat gets no arguments, it calls

cat(0) and reads file descriptor 0, the standard input, which is why it can stand on the right side of a pipe.

3.2 The system calls in the kernel

The file user/user.h lists the 22 system calls that programs can use, from fork, exit and wait to pause, uptime and sync, followed by ordinary library functions such as strcpy, atoi, printf and malloc, which run entirely in user space (Figure 3.2). The command `grep -n "sys_read\|sys_write" kernel/sysfile.c` showed that sys_read is implemented on line 69 of kernel/sysfile.c and sys_write on line 83 of the same file.

The path between the two sides is short. The function read that cat calls is a small stub generated from user/usys.pl. It puts the number of the system call into the register a7 and executes the RISC-V instruction ecall, which switches the processor into the privileged supervisor mode and enters the kernel. There, kernel/syscall.c reads the number from a7 and calls sys_read, which collects the arguments of the call and passes them to fileread.

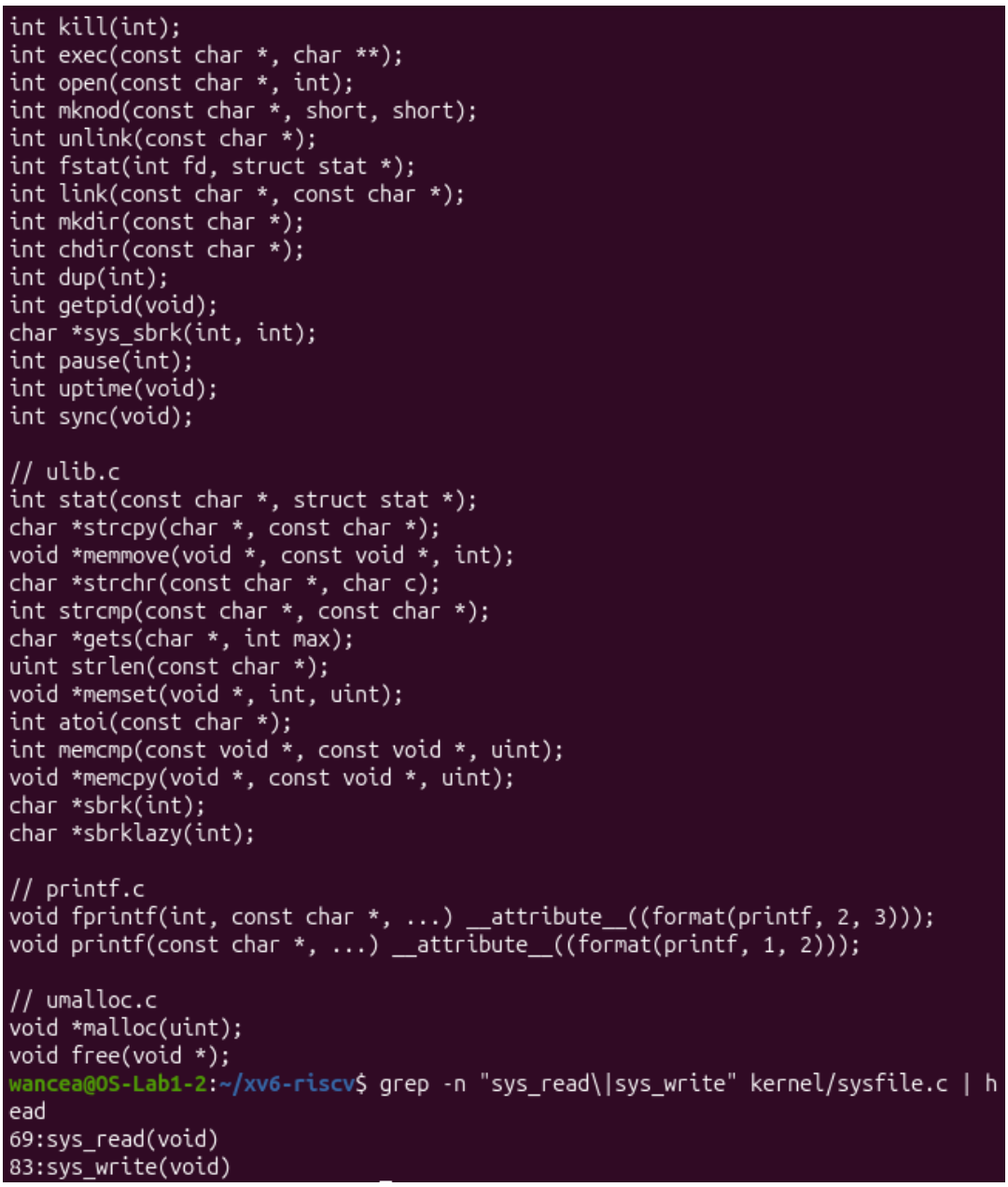


Figure 3.2 – The system calls in user/user.h and the location of sys_read and sys_write

3.3 Answers to the questions

1. user/cat.c uses five system calls: open asks the kernel to open a file and return a file descriptor, read to copy up to 512 bytes from it into the buffer, write to copy them to the descriptor 1, the standard output, close to release the descriptor, and exit to end the process with a status. fprintf, used for the error messages, also ends in write calls, on the descriptor 2.

2. sys_read is implemented in kernel/sysfile.c, on line 69 (Figure 3.2).

3. The code in kernel/ runs in the privileged mode of the processor and controls the hardware, the memory and all the processes, while the code in user/ runs in user mode, each program isolated in its own address space, and can reach files, devices or other processes only by asking the kernel through system calls.

4 WRITING THE PROGRAM SLEEP

4.1 The program and its line in the Makefile

The new program was written in the file user/sleep.c with the editor nano. The first attempt to open the file was made from the home folder instead of ~/xv6-riscv, so nano reported that the directory user does not exist, and the work continued inside ~/xv6-riscv. The program checks that it got exactly one argument. If not, it prints a usage message to file descriptor 2, the standard error, and ends with the status 1. Otherwise it converts the argument from text to a number with atoi, passes the number to the system call and ends with the status 0. The final version of the program is shown in Listing 4.1.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
```

Listing 4.1 – The program user/sleep.c

To include the program in the build, the line `$U/_sleep\` was added to the list UPROGS in the Makefile, under the line of echo, where it became line 135. The command `grep -n "_sleep" Makefile | cat -A` makes invisible characters visible and confirmed that the line starts with a tab, shown as ^I, and that the backslash is the last character before the end of the line, shown as $ (Figure 4.1). The backslash continues the list on the next line. If anything followed it, for example the comment printed next to this line in the handout, the list would end on this line and the next lines of the Makefile would become invalid.

```
wancea@OS-Lab1-2:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
wancea@OS-Lab1-2:~/xv6-riscv$ nano Makefile
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n "_sleep" Makefile | cat -A
135:^I$U/_sleep\$
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n "_sleep" Makefile | cat -A
135:^I$U/_sleep\$
```

Figure 4.1 – The first version of user/sleep.c and the new line in the Makefile

4.2 The renamed system call

The first build of the program failed with the error "implicit declaration of function 'sleep'" (Figure 4.2). xv6 is compiled with the option -Werror, which turns every warning into an error. The reason was already visible in user/user.h (Figure 3.2): the current version of xv6 has no system call named sleep. It has been renamed pause, and the program in the handout was written before this change. The kernel also has an internal function named sleep in kernel/proc.c, which blocks a process while it waits for an event, so the new name keeps the two apart. The call in sleep.c was changed to `pause(atoi(argv[1]))`, and the program kept the name sleep that the task asks for.

```
wancea@OS-Lab1-2:~/xv6-riscv$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-p
ointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/wancea/xv6-riscv=. -march=rv64gc
-std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin
-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-built
in-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-
strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free
 -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-bui
ltin-vprintf -I. -fno-stack-protector -fno-pie -no-pie   -c -o user/sleep.o user
/sleep.c
user/sleep.c: In function ‘main’:
user/sleep.c:11:3: error: implicit declaration of function ‘sleep’ [-Wimplicit-f
unction-declaration]
   11 |   sleep(atoi(argv[1]));   // system call into the kernel
      |   ^~~~~
At top level:
cc1: note: unrecognized command-line option ‘-Wno-unknown-attributes’ may have b
een intended to silence earlier diagnostics
make: *** [<builtin>: user/sleep.o] Error 1
wancea@OS-Lab1-2:~/xv6-riscv$ cd
```

Figure 4.2 – The compilation error caused by the renamed system call

The second build succeeded (Figure 4.3). The program was compiled and linked into user/_sleep, and mkfs placed it into fs.img right after echo. This time the first 1019 blocks of the

image were in use instead of 983, so the new program added 36 blocks. The executable has 35280 bytes, which fill 35 blocks of 1024 bytes. The extra block is an indirect block: an xv6 inode keeps the addresses of only the first 12 data blocks of a file (NDIRECT is 12 in kernel/fs.h), and the addresses of the other blocks of a larger file are kept in one more block.

```
wancea@OS-Lab1-2:~/xv6-riscv$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-p
ointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/wancea/xv6-riscv=. -march=rv64gc
-std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin
-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-built
in-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-
strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free
 -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-bui
ltin-vprintf -I. -fno-stack-protector -fno-pie -no-pie   -c -o user/sleep.o user
/sleep.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_sleep user/s
leep.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_sleep > user/sleep.asm
riscv64-linux-gnu-objdump -t user/_sleep | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^
$/d' > user/sleep.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_sleep user/_forktest user/_gr
ep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_s
tressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_
forphan user/_dorphan user/_sync
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1
953 total 2000
balloc: first 1019 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp
3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,f
ormat=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

Figure 4.3 – The successful build with the new program sleep

4.3 Testing sleep inside xv6

Inside xv6, the command ls listed sleep as a file with the inode number 5 and a size of 35280 bytes (Figure 4.4). The command `sleep 10` paused and then returned to the prompt, and `sleep` without an argument printed the usage message. One tick of xv6 is the interval of the timer interrupt, which kernel/trap.c sets to 1000000 cycles of the timer, about a tenth of a second, so `sleep 10` waits about one second. In the kernel, sys_pause in kernel/sysproc.c reads the tick counter and puts the process to sleep until the counter has grown by the requested number of ticks.

The first ls in Figure 4.4 failed with "exec ls failed". The command had been pasted into the terminal, and the pasted text most likely reached xv6 together with invisible control characters, so the shell looked for a program with a different name. Typed by hand, the same command worked.

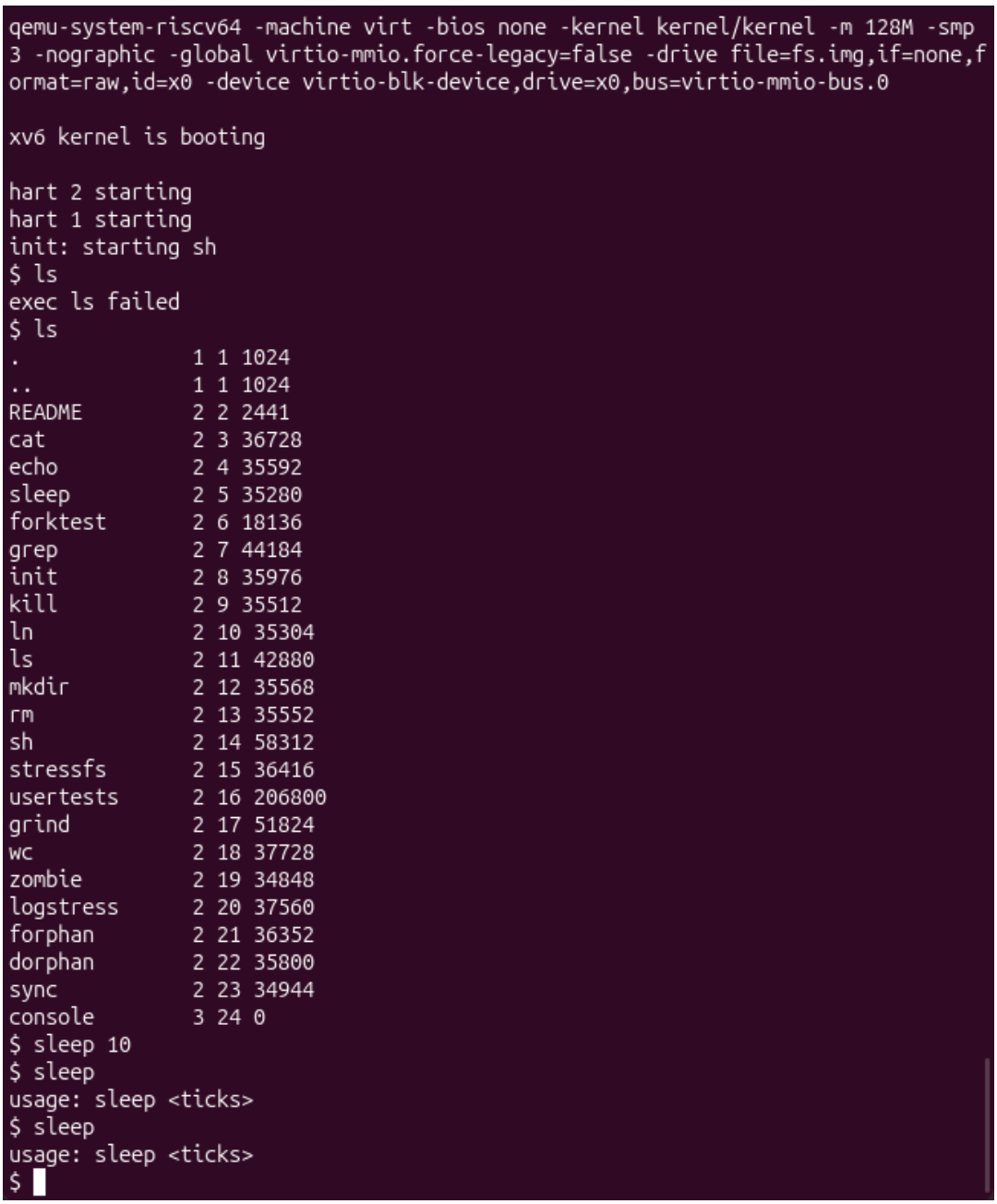


Figure 4.4 – Testing the program sleep inside xv6

The same file sleep.c was built again on 25 October 2026, after the tree had been updated to the newest version of xv6 (commit f780d78 of 15 October 2026), and it still compiles and runs without changes; this check is shown in the report of the eighth laboratory work.

4.4 The path of the system call

Table 4.1 follows the call that the program sleep makes, from the user program down to the kernel.

Table 4.1 – The path of the pause system call through xv6

Step	What happens	File
Program	main calls `pause(ticks)` like an ordinary function	`user/sleep.c`
Stub	puts 13, the number of SYS_pause, into the register a7 and executes `ecall`	`user/usys.S`
Dispatch	the kernel reads a7 and calls the matching function from its table of system calls	`kernel/syscall.c`
Service	sys_pause waits until the tick counter has grown by the requested value	`kernel/sysproc.c`

The program sleep never touches the timer itself. It only asks, and the kernel, which owns the timer and the scheduler, does the waiting and lets other processes use the processor in the meantime.

CONCLUSIONS

The work moved from watching an operating system from the outside to building one. xv6 was compiled with a cross compiler, because it runs on RISC-V and not on the x86_64 processor of the host, and it was booted in QEMU on an emulated RISC-V machine. Its small set of programs behaves like the Linux commands of the first work, and its source shows both sides of every system call: the program in user/ that asks and the code in kernel/ that answers. Adding the program sleep showed the whole path of one system call, from the call in C through the ecall instruction and the table of system calls to sys_pause in the kernel.

Most problems of this work came from differences between the handout and the current software. The tools were not installed in advance, the RISC-V emulator has a different package name on Ubuntu 26.04, and the system call sleep has been renamed pause in the current xv6. The last one was solved by reading user/user.h, the same file that the third part of the work asks to read, which shows that the source code is often the fastest way to understand an error. The work also ran on three levels at once: xv6 ran in QEMU on Ubuntu, and Ubuntu ran in VirtualBox on Windows, so three operating systems were managing resources at the same time, each one inside the resources given to it by the one below.

REFERENCES

1. Meet the OS You Will Build. Laboratory Work 2, Operating Systems course. Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova, 2026.
2. xv6 for RISC-V. Source code repository, MIT PDOS [online]. Available: https://github.com/mit-pdos/xv6-riscv